

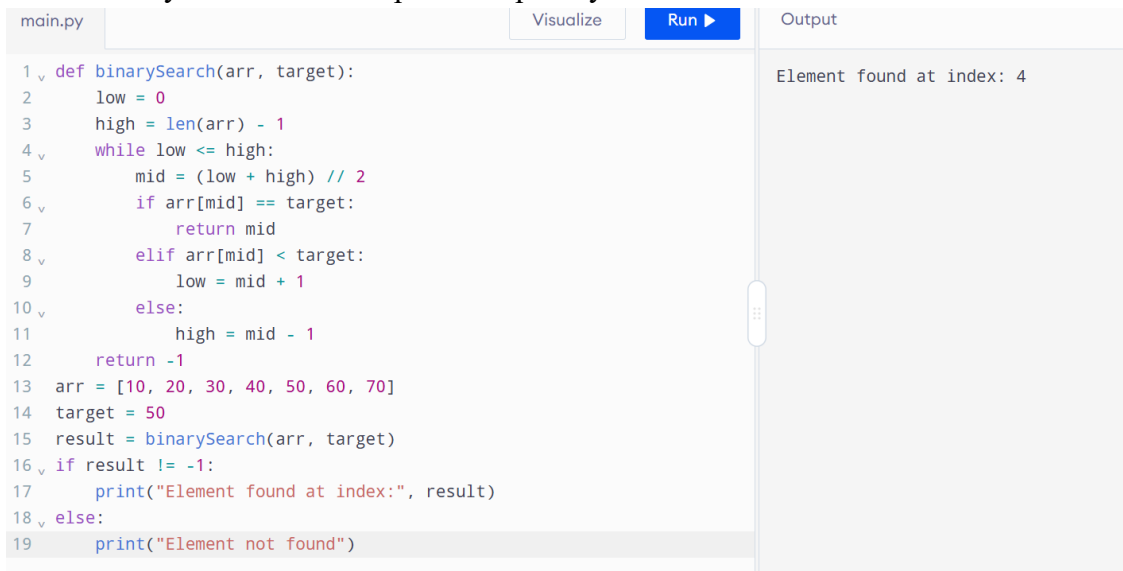
4. Binary Search with Big-O Analysis

Aim

To implement **Binary Search using Python** to search for an element in a sorted array and analyze its time and space complexity using Big-O notation.

Procedure

1. Create a **sorted array** of elements.
2. Set **low** to the first index and **high** to the last index.
3. Calculate the middle index using $(low + high) // 2$.
4. Compare the middle element with the target element.
5. If they are equal, the element is found.
6. If the target is greater than the middle element, search the right half.
7. If the target is smaller than the middle element, search the left half.
8. Repeat the process until the element is found or the search range becomes empty.
9. Display the result.
10. Analyze the time and space complexity.



The screenshot shows a code editor with a file named 'main.py'. The code implements a binary search function. The array is [10, 20, 30, 40, 50, 60, 70] and the target is 50. The output window shows 'Element found at index: 4'.

```
1 def binarySearch(arr, target):
2     low = 0
3     high = len(arr) - 1
4     while low <= high:
5         mid = (low + high) // 2
6         if arr[mid] == target:
7             return mid
8         elif arr[mid] < target:
9             low = mid + 1
10        else:
11            high = mid - 1
12    return -1
13 arr = [10, 20, 30, 40, 50, 60, 70]
14 target = 50
15 result = binarySearch(arr, target)
16 if result != -1:
17     print("Element found at index:", result)
18 else:
19     print("Element not found")
```

Output: Element found at index: 4

Result

Thus, the Python program successfully searches for the given element using the **Binary Search algorithm**.

